AN ILLUSTRATED GUIDE TO AI AGENTS · CHAPTER 2

Large Language Models

The engine behind every agent: tokens, training and the Transformer

Maarten Grootendorst & Jay Alammar · Chapter 2 summary

What this chapter covers

PART 1

What agent developers should know

Core intuitions, capabilities and limitations, without needing the deep machinery.

- Tokens, system prompts and tool use
- Building the LLM into TinyAgent
- Pre-training, SFT and reinforcement learning
- Transformer overview, context length, KV cache

PART 2

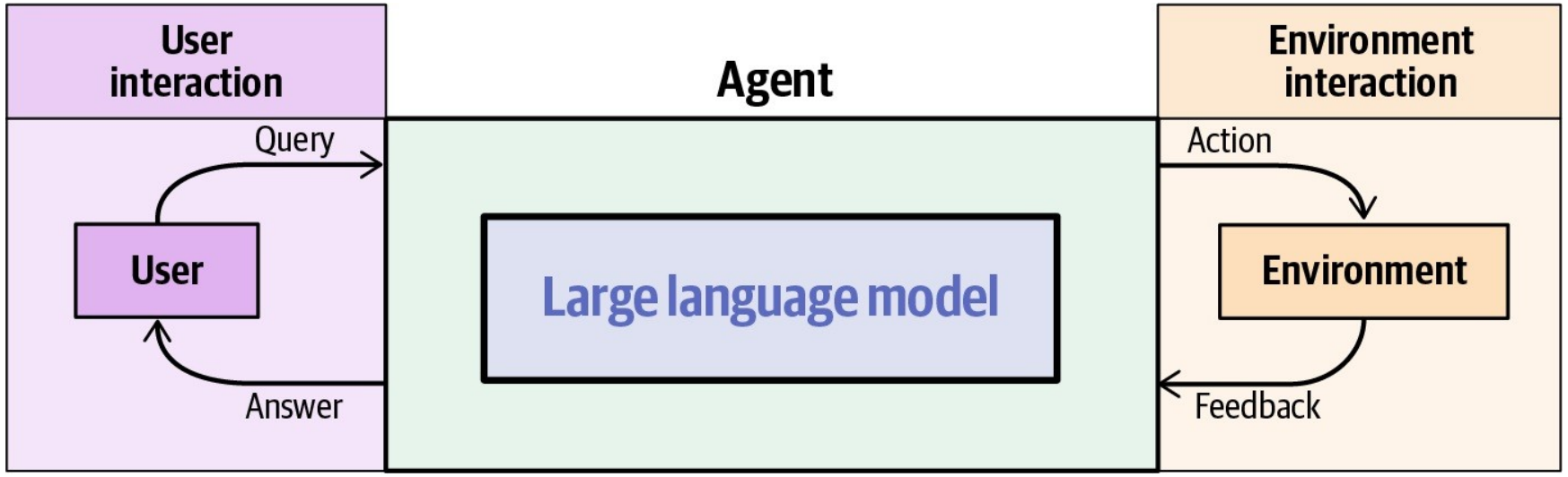
A deeper dive into LLMs

Model internals, useful for diagnosing agent failures and selecting models.

- How self-attention works (queries, keys, values)
- KV cache revisited
- Efficient attention: MLA and DSA
- Mixture of experts

Building agents? Part 1 gives you everything you need for the next chapter.

The LLM bridges the user and the environment



1 · Understand
Process the user's query

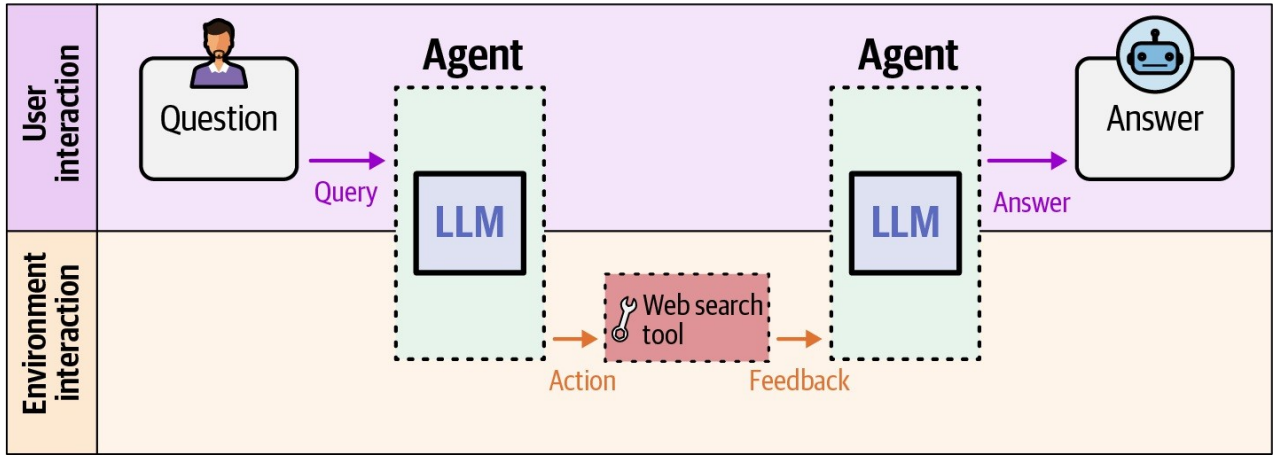
2 · Act
Choose the right action

3 · Observe
Process the feedback from the action

4 · Reply
Communicate back to the user

Figure 2-1 · An Illustrated Guide to AI Agents

An information-seeking agent, step by step



1 The user asks the agent a question.

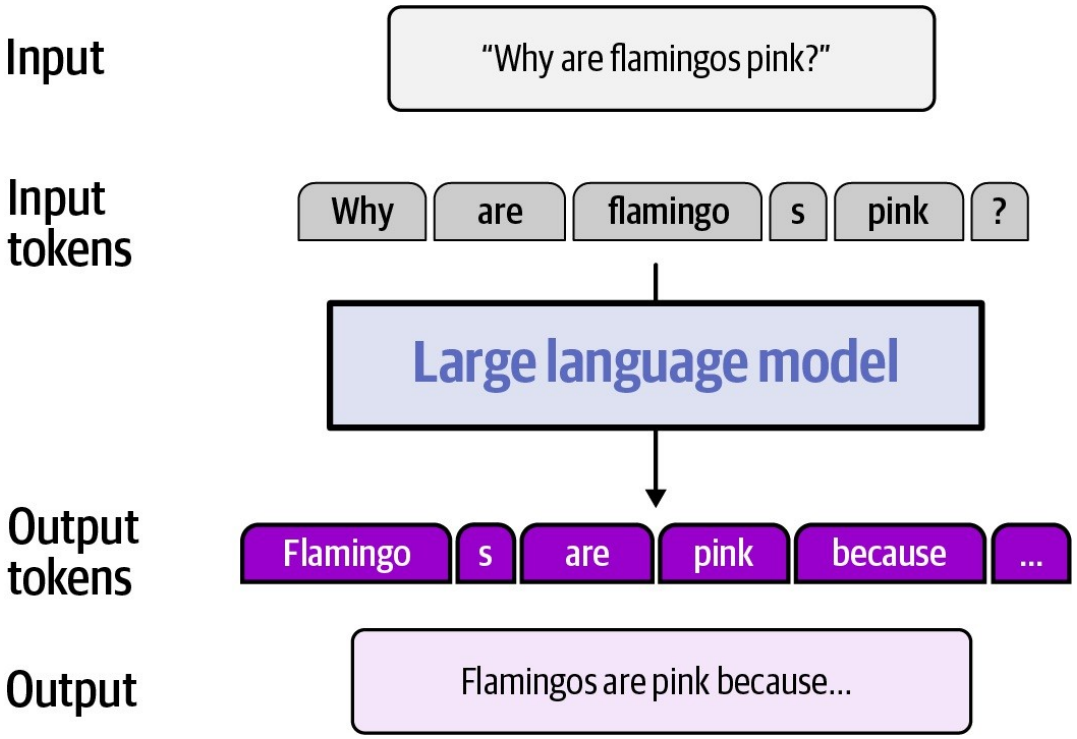
2 The agent calls a web search tool, pulling information in from the environment.

3 It decides it now has enough information to answer.

4 The agent prints its answer to the user.

Figure 2-2 · An Illustrated Guide to AI Agents

Input and output tokens



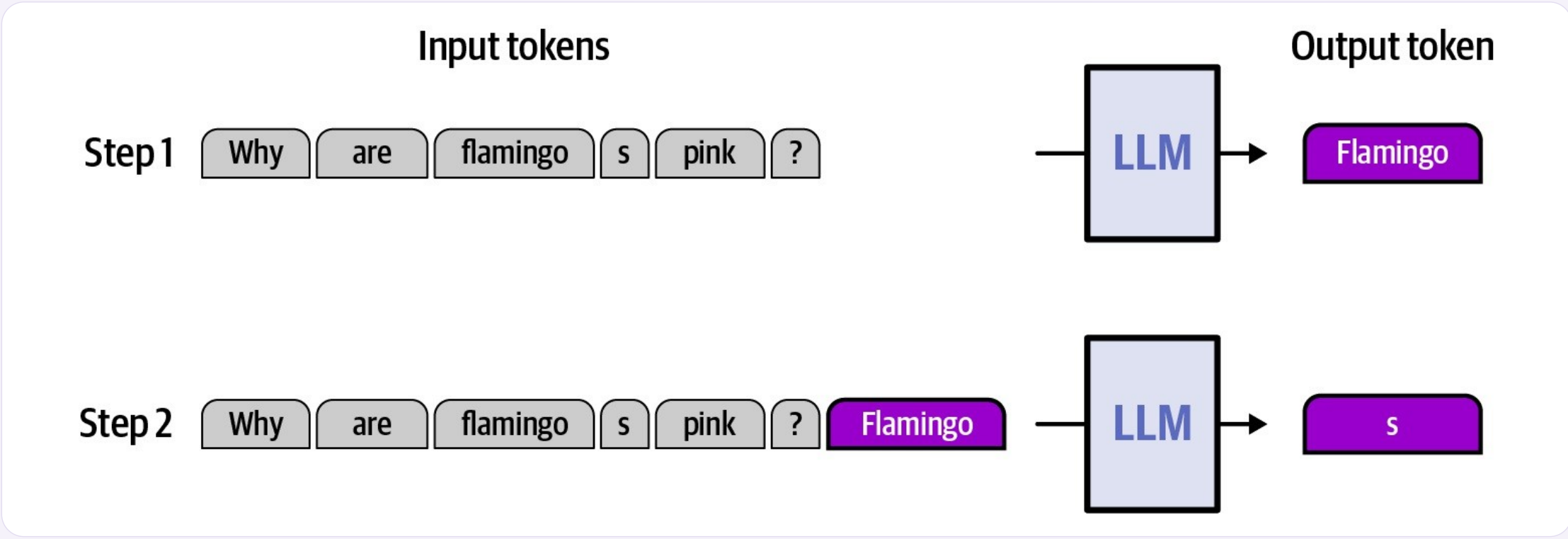
Tokens can be words, parts of words, numbers or punctuation. "flamingos" becomes "flamingo" and "s".

Input can be a user query or feedback from the environment.

Output is generated token by token until the response is complete.

Figure 2-3 · An Illustrated Guide to AI Agents

Autoregression: each output feeds the next step



Generating text is a loop: produce a token, add it to the input, process again. Models that consume their own output are called autoregressive.

Figure 2-4 · An Illustrated Guide to AI Agents

From language model to agent engine

A base language model becomes an agent's brain through post-training, which teaches it formats that support three capabilities.

System prompt

A privileged input from the deployer that shapes behavior before the model sees a single user token.

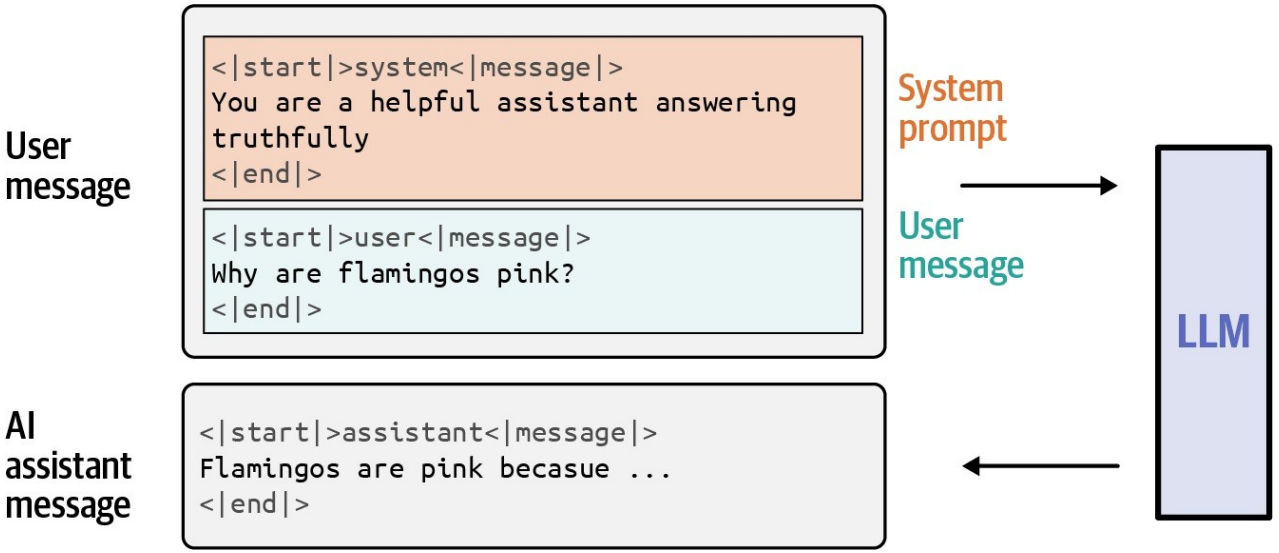
Multi-turn chat

Tracks who said what across a longer conversation, one way to give the LLM memory (Chapter 4).

Tool use

The LLM outputs a format that picks a function to invoke; the agent software calls it (Chapter 5).

The system prompt



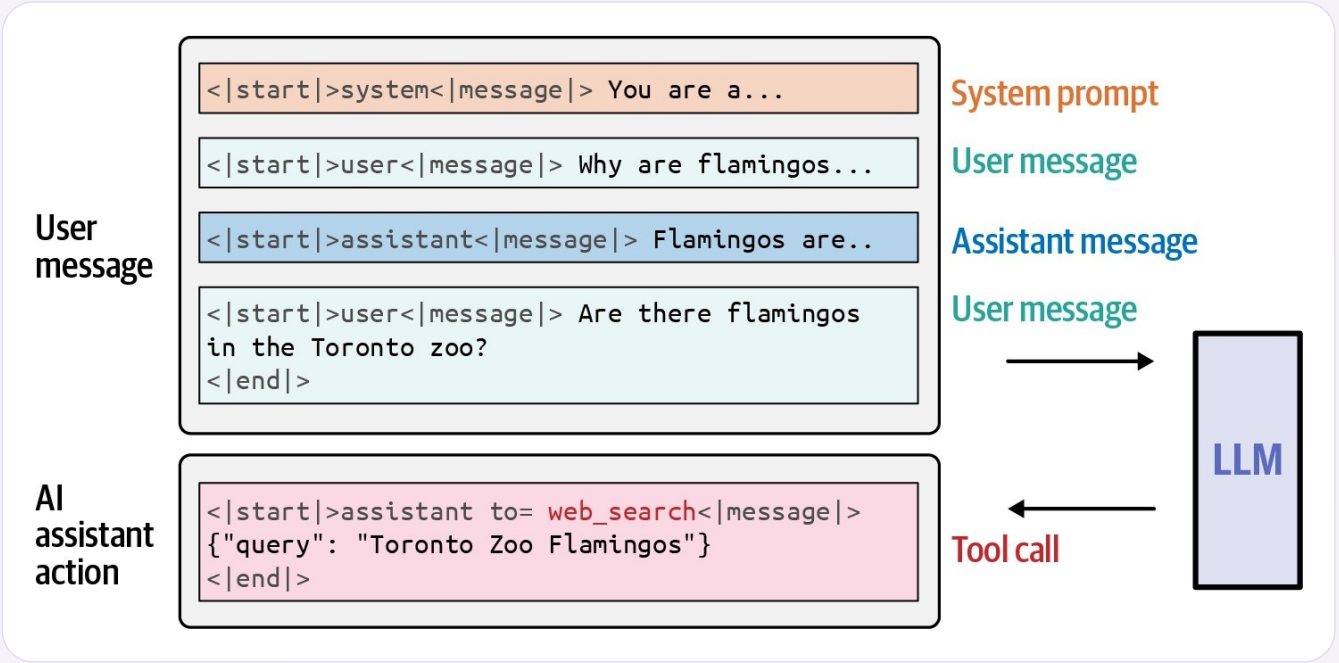
Privileged. Takes precedence over what end users ask.

No retraining. Part of how the model is used, so deployers customize behavior without touching the model.

The format shown is simplified from OpenAI Harmony, used by GPT-OSS.

Figure 2-5 · An Illustrated Guide to AI Agents

Tool use as text in and out



Structured call. The LLM emits web_search with a query instead of an answer.

Software executes. The agent wrapper spots the pattern and calls the function.

Listed up front. Available functions are usually described in the system prompt.

Figure 2-6 · An Illustrated Guide to AI Agents

TinyAgent: choosing the LLM

Gemma 3 12B

No native reasoning or tool calling: shows prompt-driven behavior

Gemma 4 E4B

Trained for agentic tasks, effectively 4 billion parameters: shows model-driven behavior

Building reasoning and tool calling yourself with prompts shows how newer models learned them natively.

OpenAI-compatible endpoint

`base_url`	URL of your hosted LLM
`api_key`	Key for accessing it
`model`	Name of your LLM
`messages`	The query to send

Hosted locally with Ollama at localhost:11434/v1; LMStudio and llama.cpp also work.

Response, Step and Trajectory

Response

What one LLM call returns.

```
content
reasoning
tool_call
metadata
```

Step

One thing the agent did; the answer ends its turn.

```
thought
action
observation
answer
```

Trajectory

The sequence of steps: a record for finding where and why an agent failed.

```
runs
  query
  steps
```

An action is a tool call. Thought, action, observation loops come in Chapter 6.

Giving TinyAgent its brain

```
def _step(self, task: str) -> str:
    messages = [{"role": "user", "content": task}]
    response = self.llm.generate(messages)
    self.trajectory.add(response)
    return response.content
```

```
agent.run("What is 2 + 2?")
"2 + 2 is 4."
```

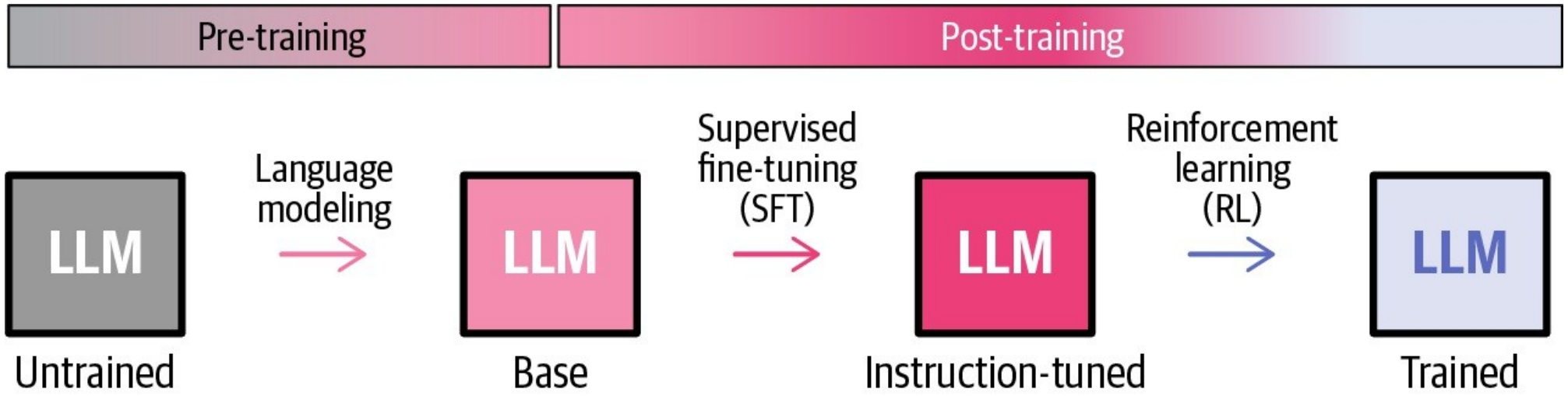
WHAT WE BUILT

agent.py updated: the LLM is integrated

llm.py new: wrapper for local or cloud models

trajectory.py new: Step and Trajectory tracking

Two phases of LLM training



Pre-training

Language modeling on vast data produces a base model

Supervised fine-tuning

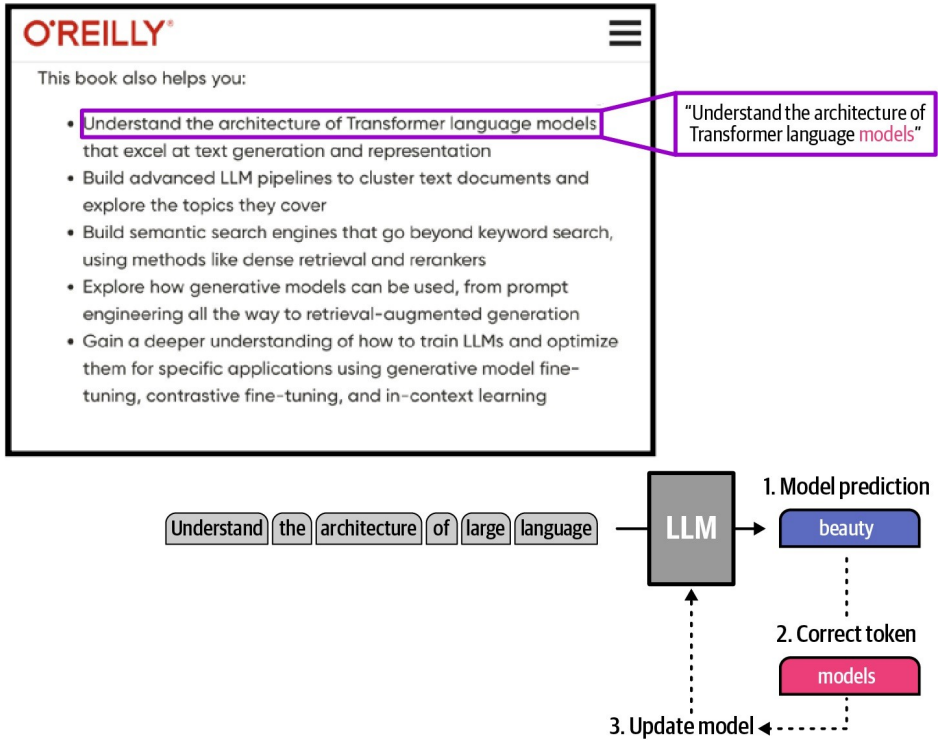
Prompt and completion pairs teach instruction following

Reinforcement learning

Reward scores push toward better responses

Figure 2-7 · An Illustrated Guide to AI Agents

Pre-training: next-token prediction



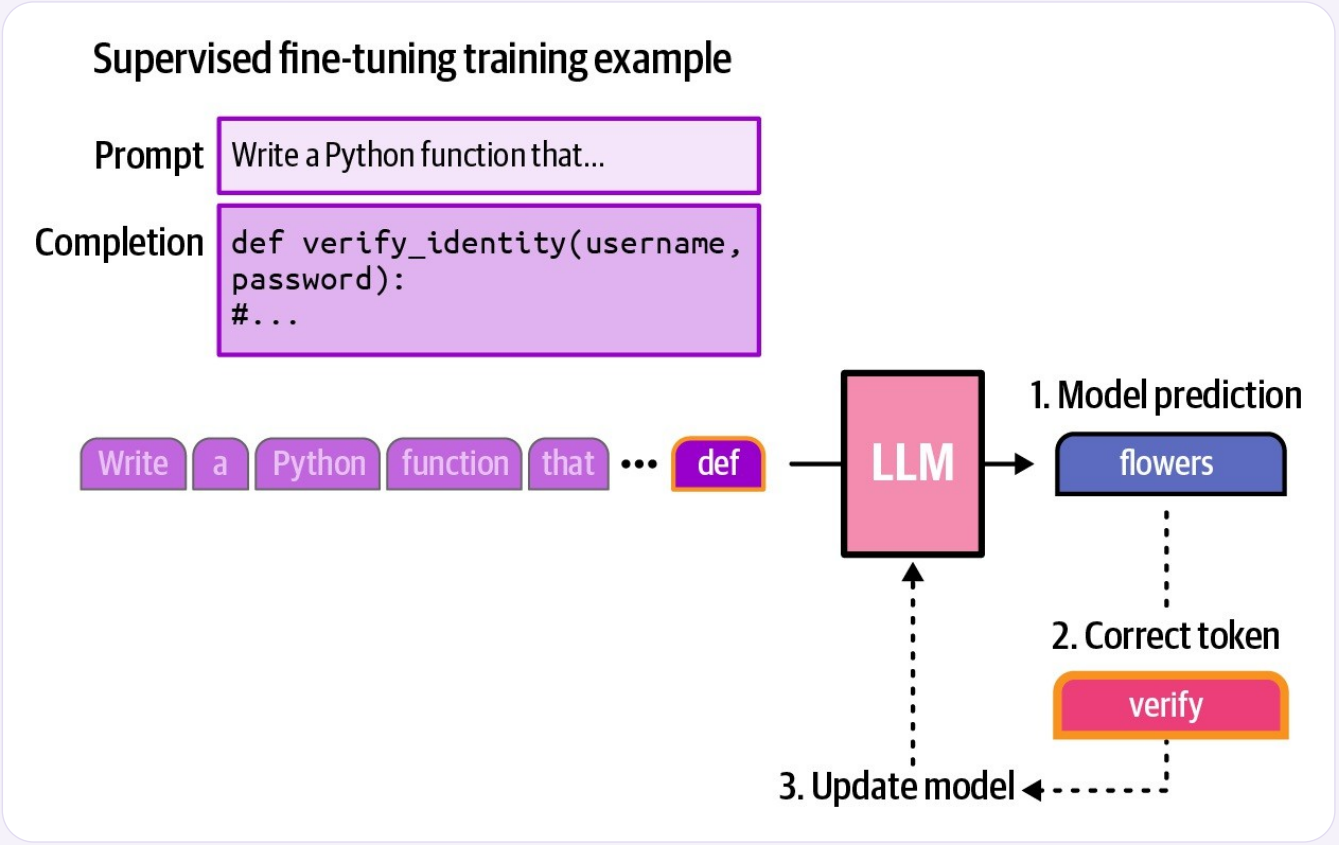
The task. Show the model a span of tokens, hide the last one, and ask it to predict it.

The update. Compare the guess with the correct token, compute the loss and adjust the weights.

The scale. Repeated billions of times on web text, books and code, this yields a base model such as GPT-3 (2020).

Figure 2-8 · An Illustrated Guide to AI Agents

Supervised fine-tuning (SFT)



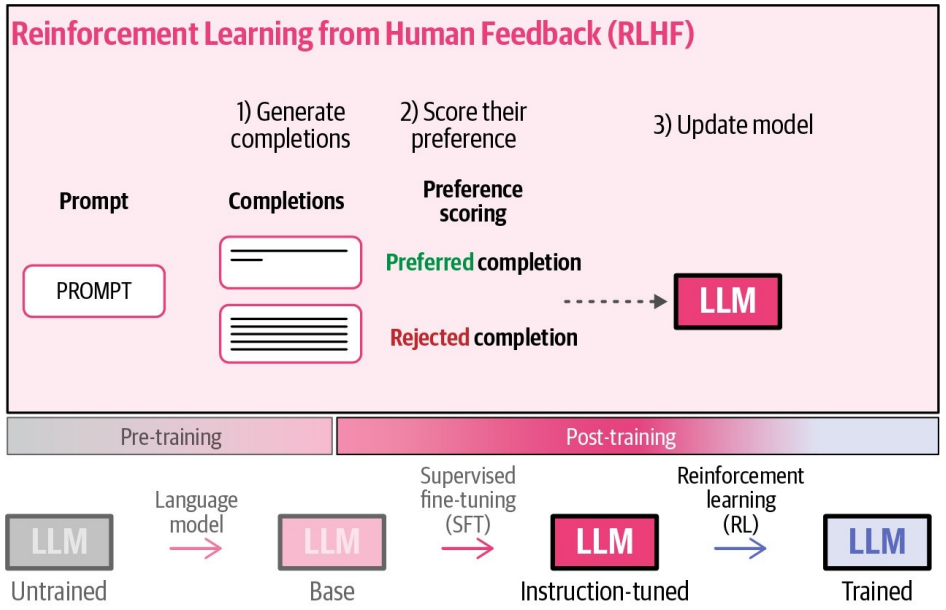
Instruction tuning. Train on prompt and completion pairs that show the desired behavior.

Loss on the completion only. Prompt tokens are context, excluded from the loss.

Essential, but it cannot fully capture what makes a response helpful, accurate or well-reasoned.

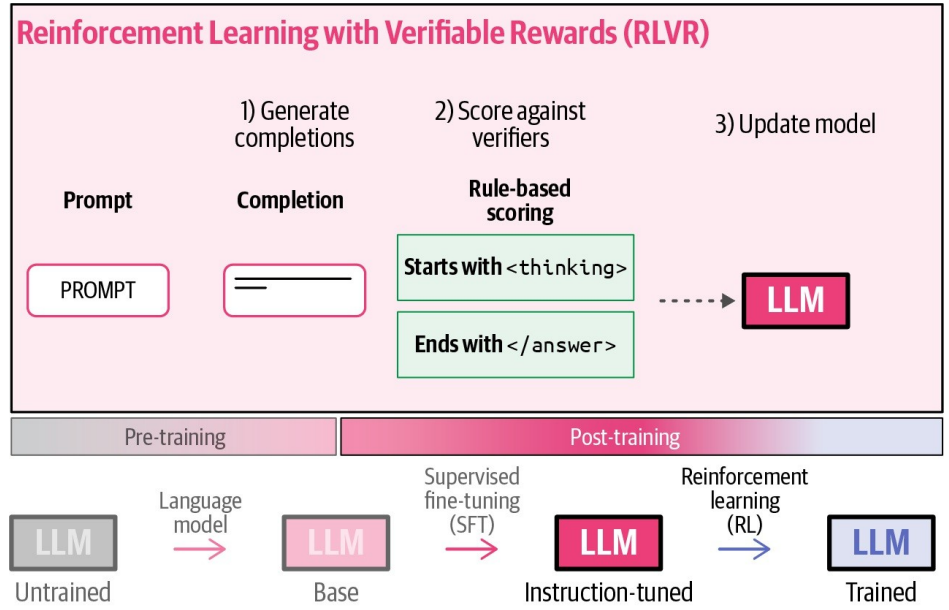
Figure 2-9 · An Illustrated Guide to AI Agents

Reinforcement learning: two kinds of reward



RLHF: human feedback

Humans or a reward model prefer one completion over another; the model learns toward the preferred one.

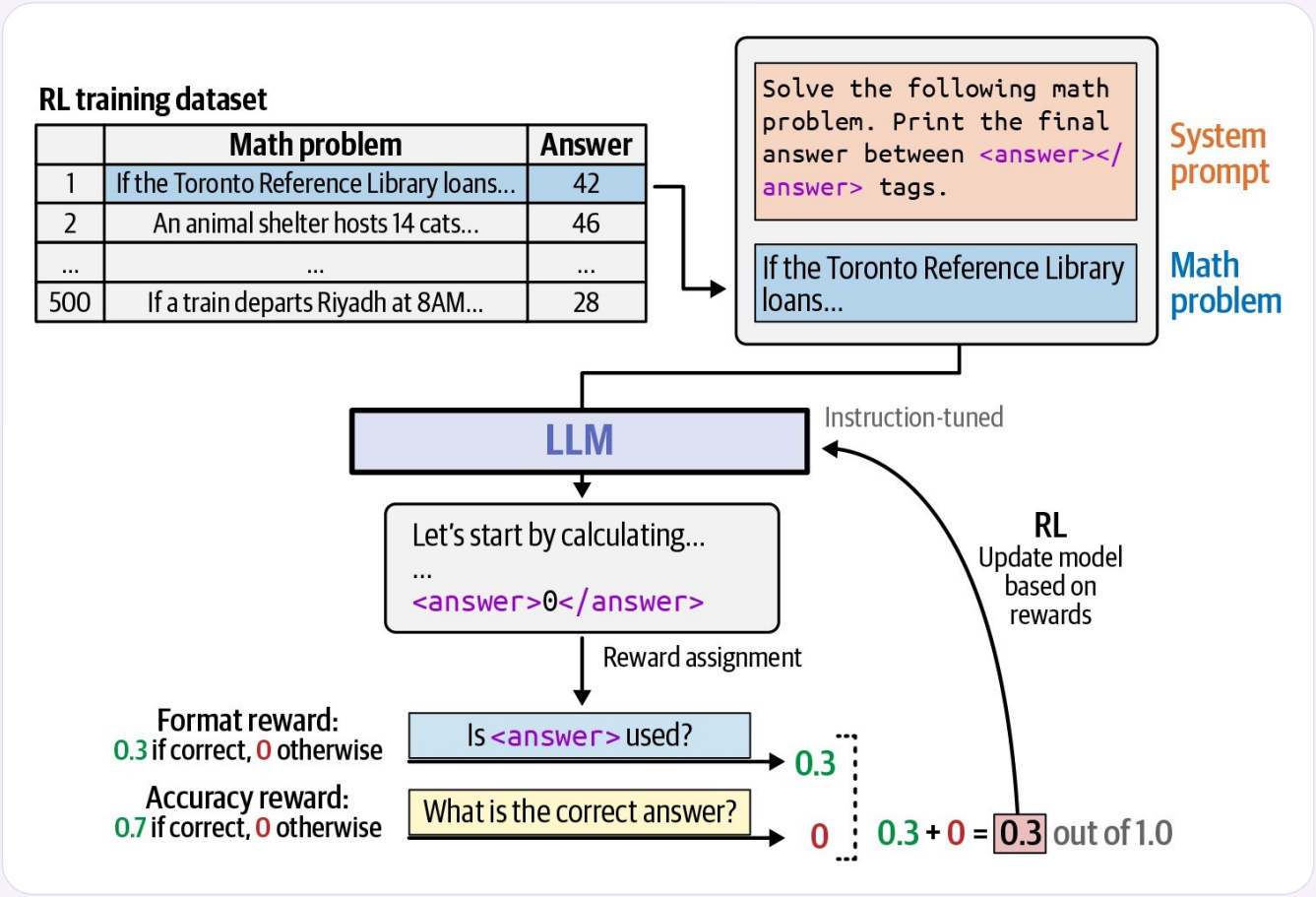


RLVR: verifiable rewards

Automated verifiers check format or the correct final answer, with no human preference needed.

Figures 2-10 and 2-11 · An Illustrated Guide to AI Agents

RLVR example: math word problems



Format reward: 0.3

Did the model use the answer tags?

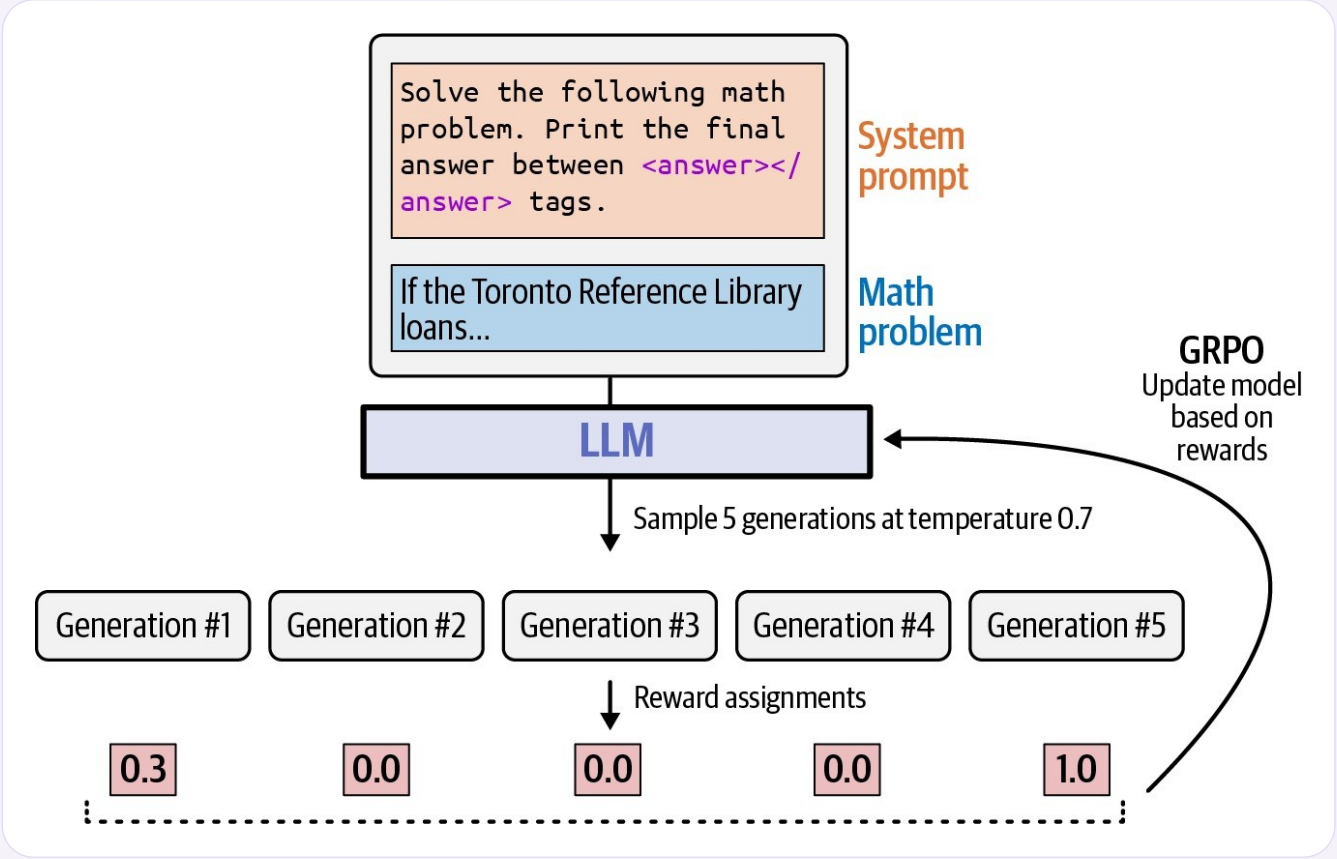
Accuracy reward: 0.7

Does the final answer match the known one?

The combined reward updates the weights. Chapter 3 shows this method training DeepSeek-R1.

Figure 2-12 · An Illustrated Guide to AI Agents

GRPO: rewards relative to a group



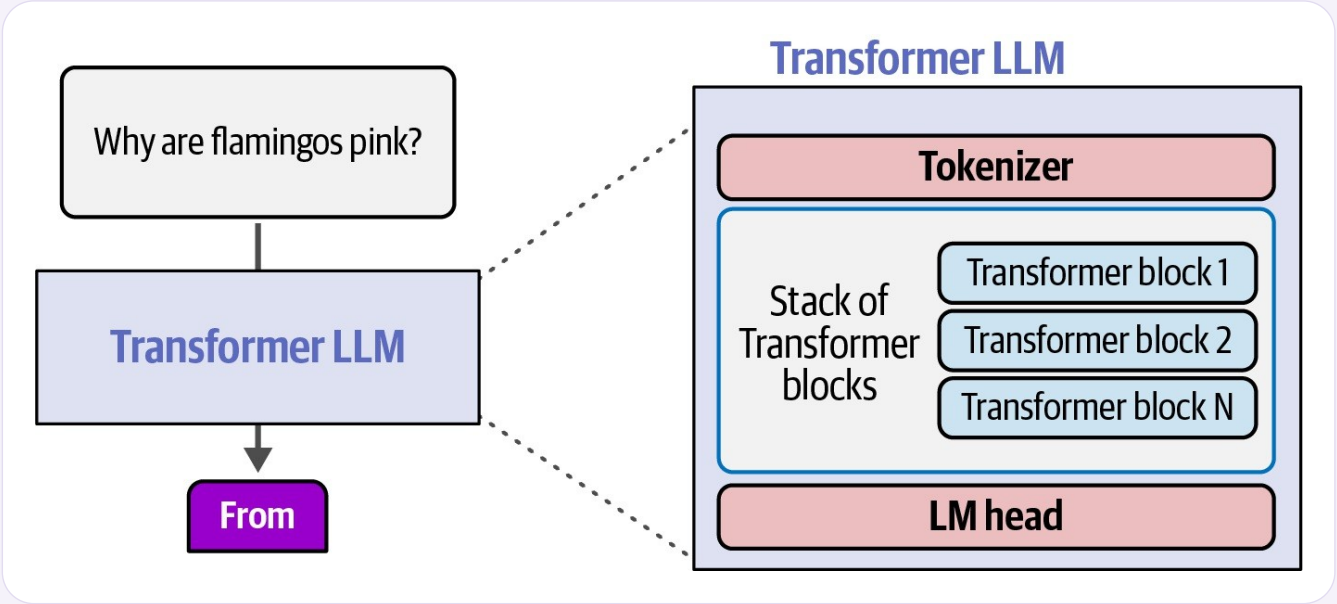
A group per prompt. Sample several responses, here five, at a temperature that keeps them diverse.

Relative scoring. Rewards are computed against the group.

Reinforce winners. Responses that beat the others are pushed up.

Figure 2-13 · An Illustrated Guide to AI Agents

The Transformer: three components



Tokenizer

Breaks text into tokens; usually supplied by the model provider

Transformer blocks

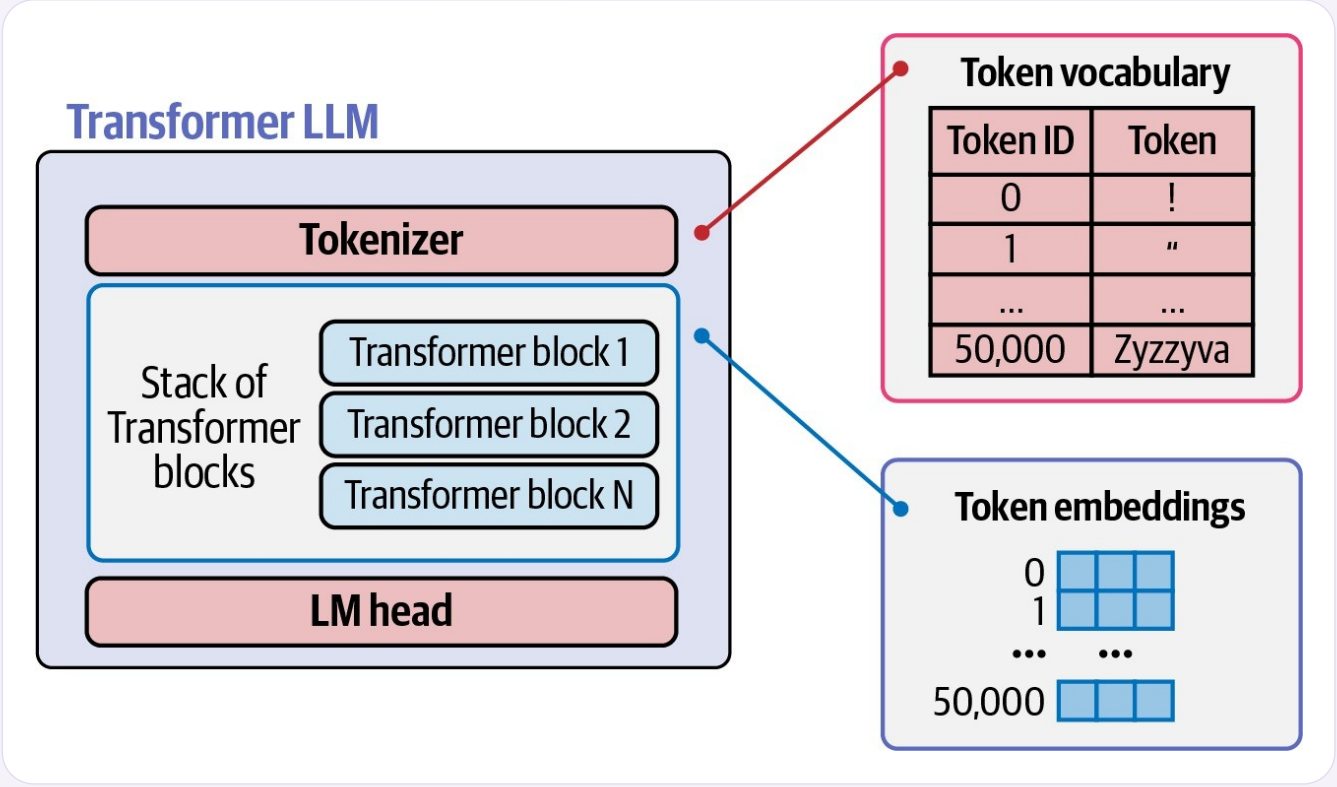
Almost all the processing happens here

LM head

Turns the final vector into a score for every token

Figure 2-14 · An Illustrated Guide to AI Agents

Tokenizer: vocabulary and embeddings



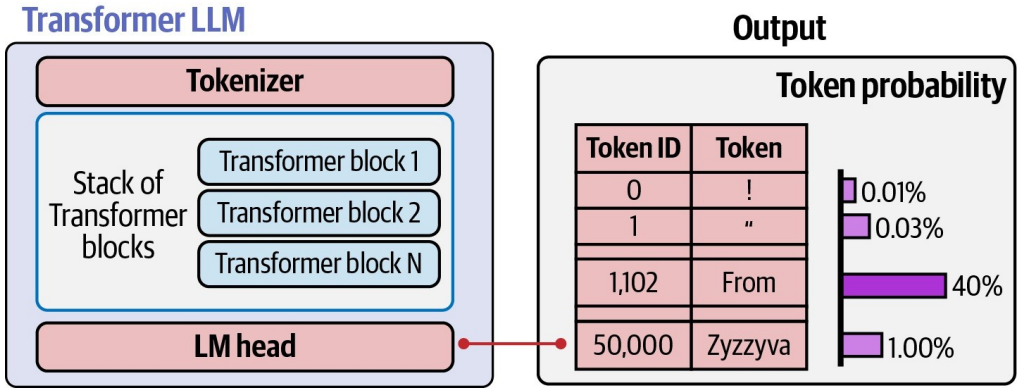
Fixed vocabulary. A set number of tokens, for example 50,000.

One vector per token. The embedding table holds a numeric vector for every token ID.

Numbers in, numbers out. The model computes with these vectors, not with words.

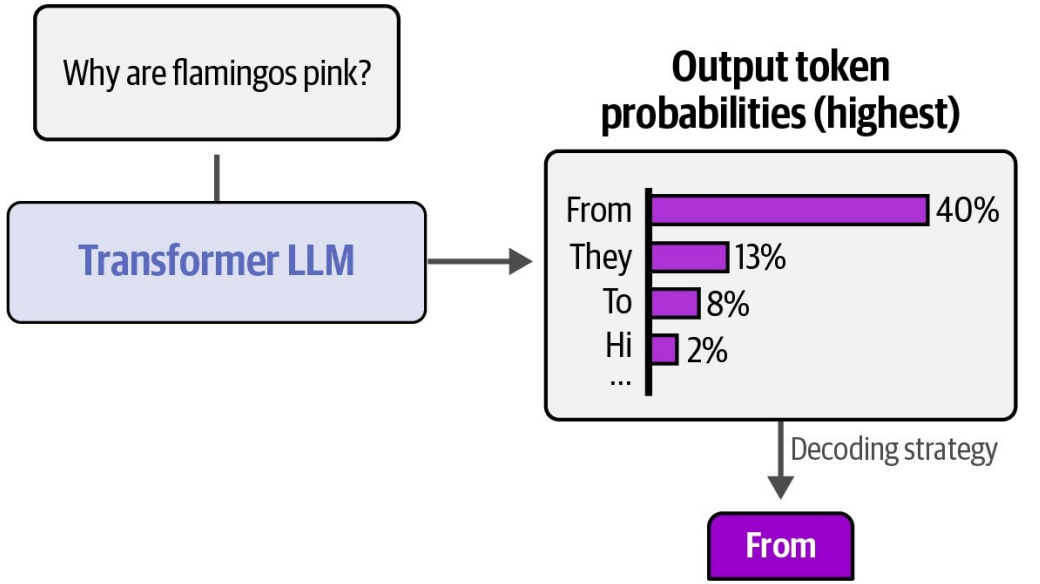
Figure 2-15 · An Illustrated Guide to AI Agents

LM head and decoding strategy



LM head

Gives every token in the vocabulary a probability of coming next.

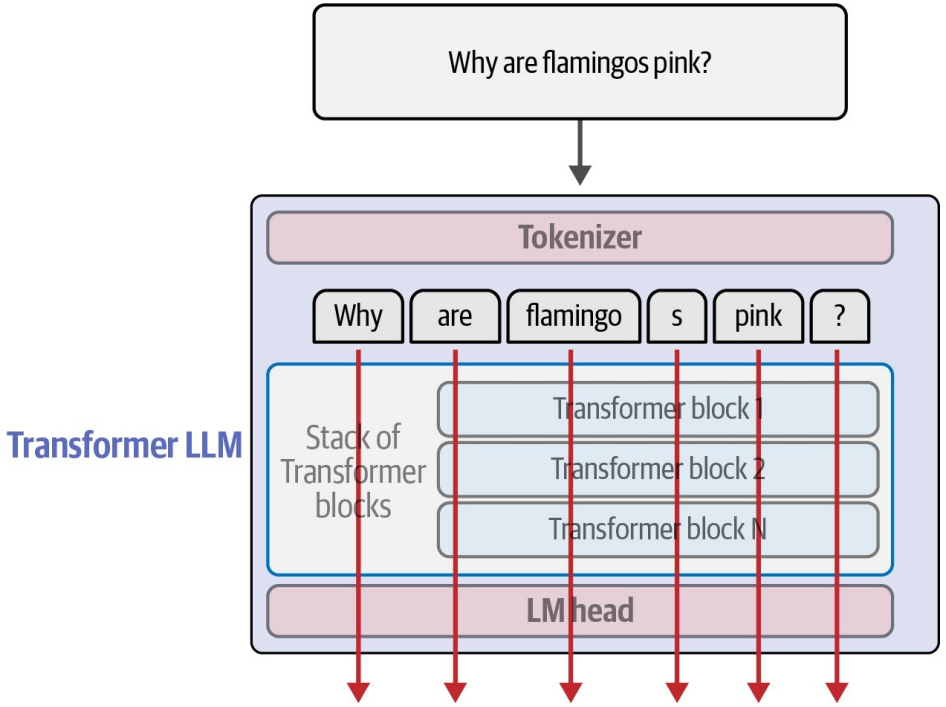


Decoding strategy

Temperature 0 always picks the top token; higher temperatures sample lower-probability tokens more often.

Figures 2-16 and 2-17 · An Illustrated Guide to AI Agents

Each token flows through its own track



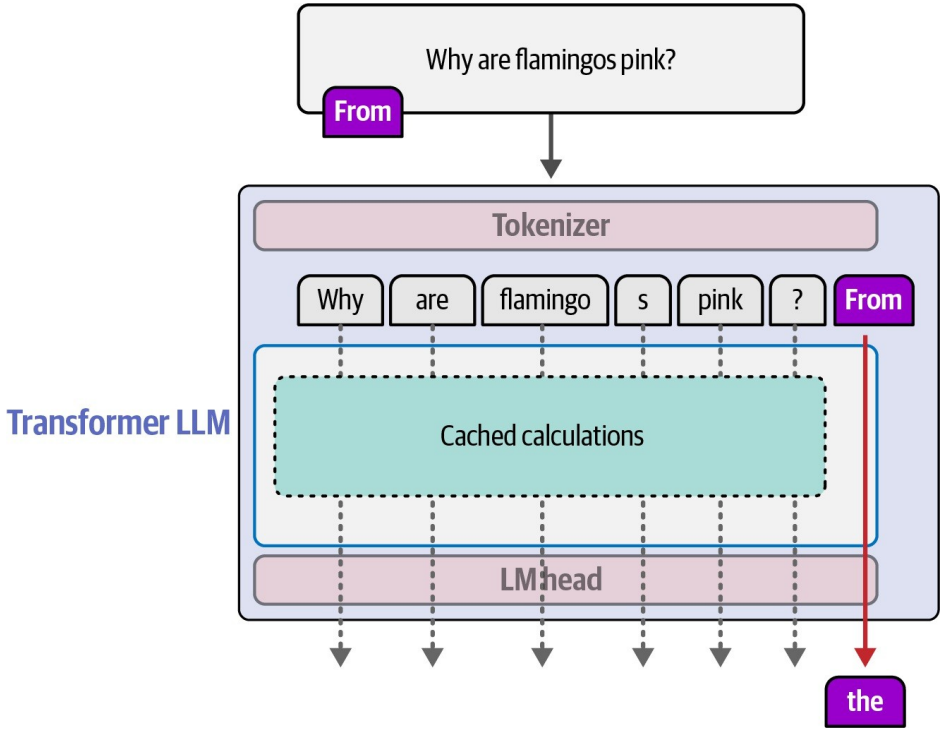
Forward pass. Block 1 hands its result to block 2, and so on, until the LM head.

Only the last track counts. The final token's output predicts the next token, informed by all earlier tokens.

Context size. The number of tracks a model supports. At 100,000, input and output are limited to that many tokens.

Figure 2-18 · An Illustrated Guide to AI Agents

The KV cache: compute once, reuse



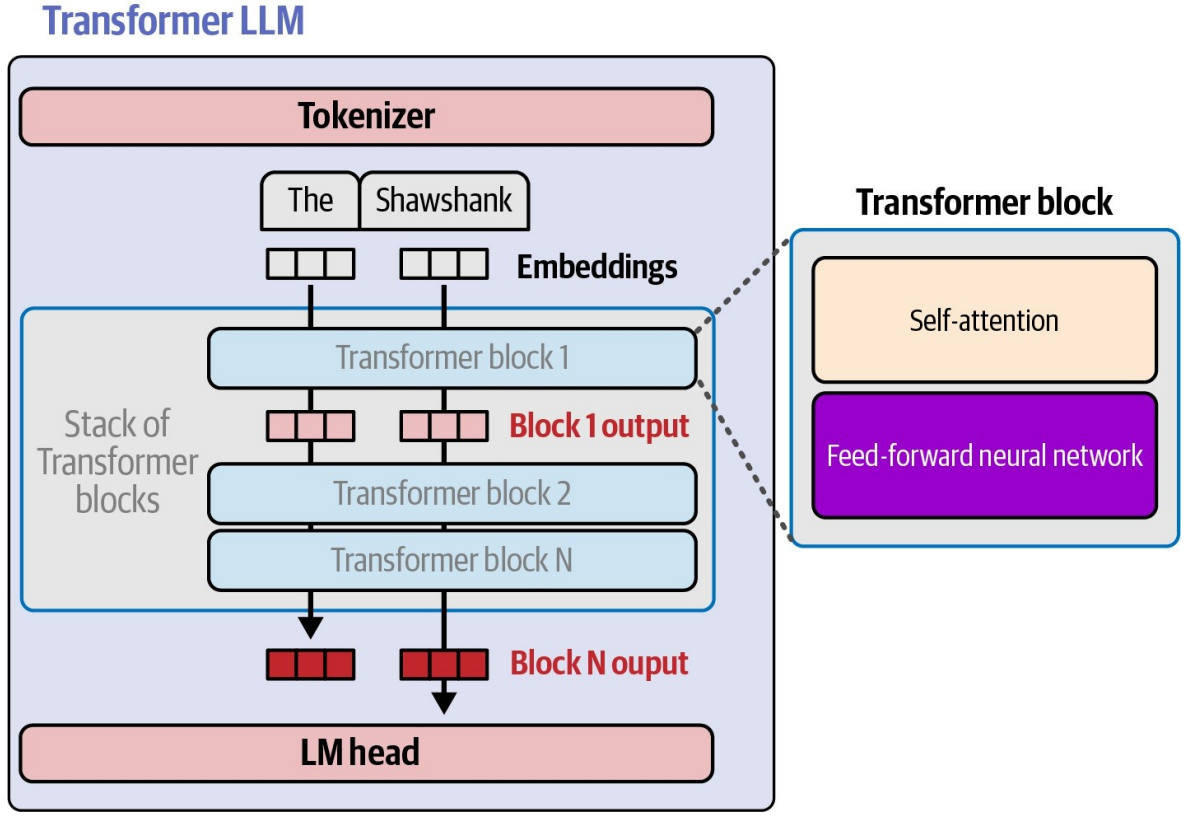
Cache the first pass. After all input tokens are processed, keep the results.

One active track. Later steps compute only the new token, reusing cached tracks.

Faster and cheaper. Also called prompt-caching or prefix-caching. Optimizing for it cuts agent latency and cost (Chapter 10).

Figure 2-20 · An Illustrated Guide to AI Agents

Inside a Transformer block



Self-attention

Gathers context from other positions in the sequence

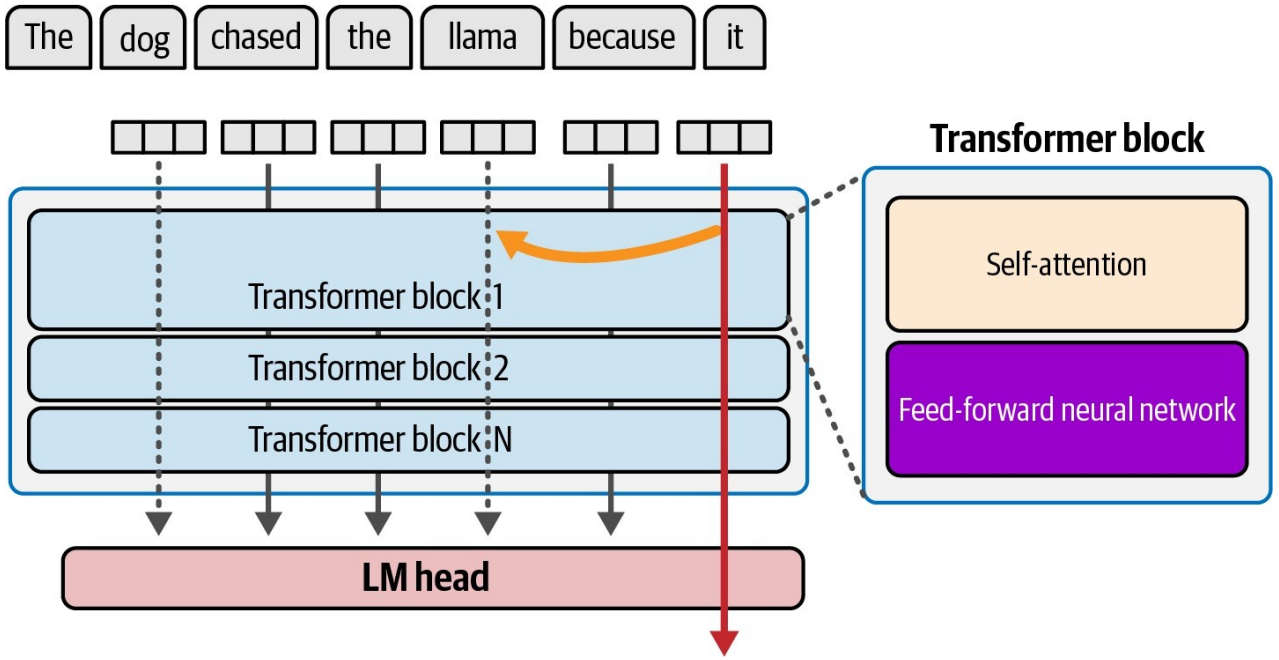
Feed-forward network

Does the heavy lifting of next-token prediction and stores factual associations, such as “The Shawshank” leading to “Redemption”

Each block takes in and hands on the same number and size of vectors.

Figure 2-22 · An Illustrated Guide to AI Agents

Self-attention resolves context



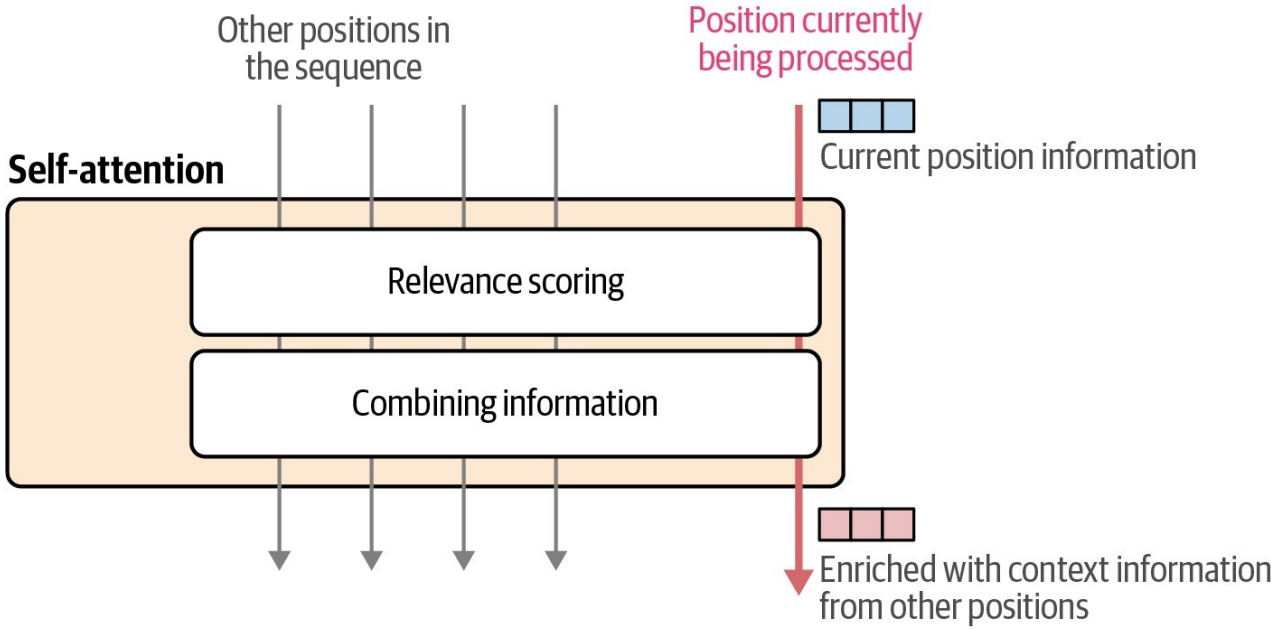
Word order and context matter. “Bank” can be a riverbank or a financial institution.

Look back. In “The dog chased the llama because it”, attention links “it” to the earlier word.

Learned relevance. Which previous tokens matter is learned in training.

Figure 2-26 · An Illustrated Guide to AI Agents

Self-attention works in two steps



1 · Relevance scoring

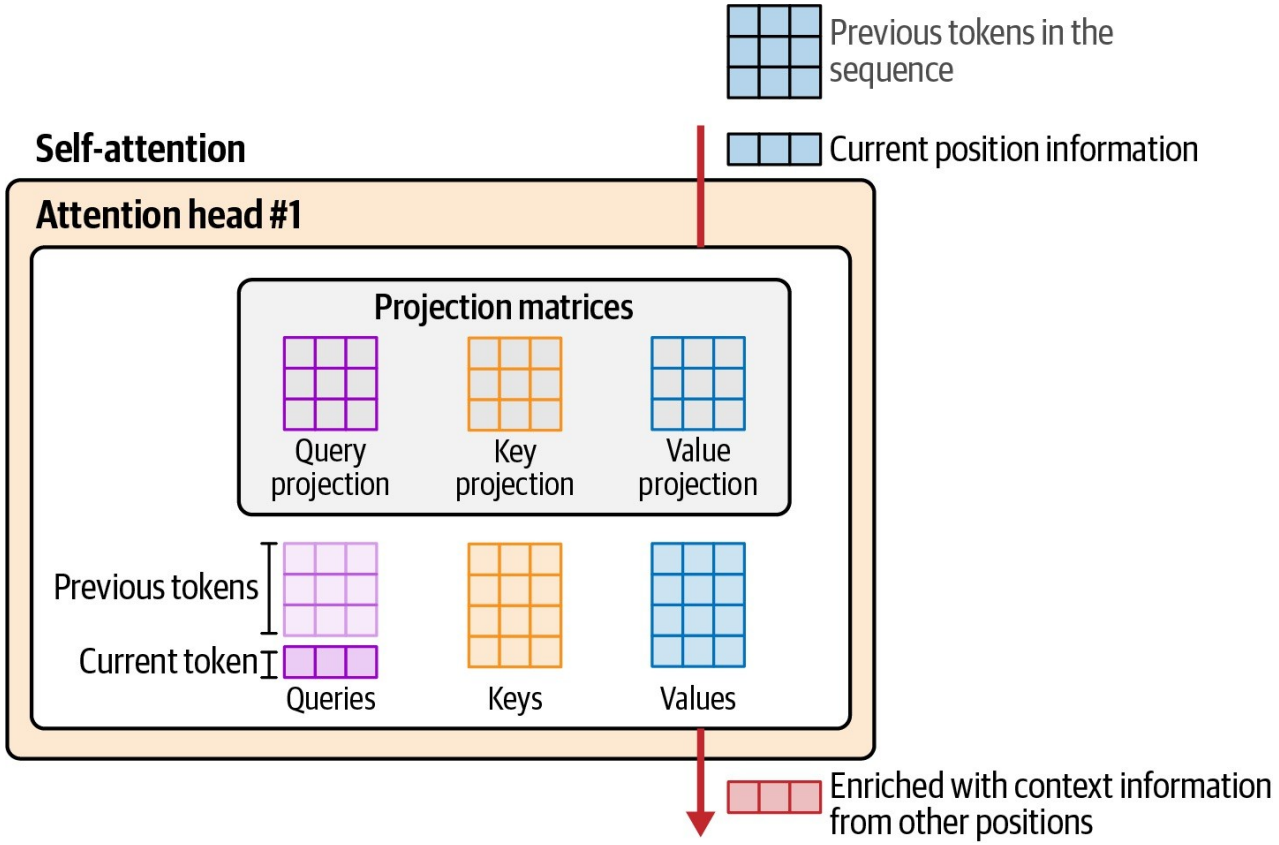
How much attention to pay to each preceding position

2 · Combining information

Blend those positions into the current vector in proportion to their scores

Figure 2-28 · An Illustrated Guide to AI Agents

Queries, keys and values



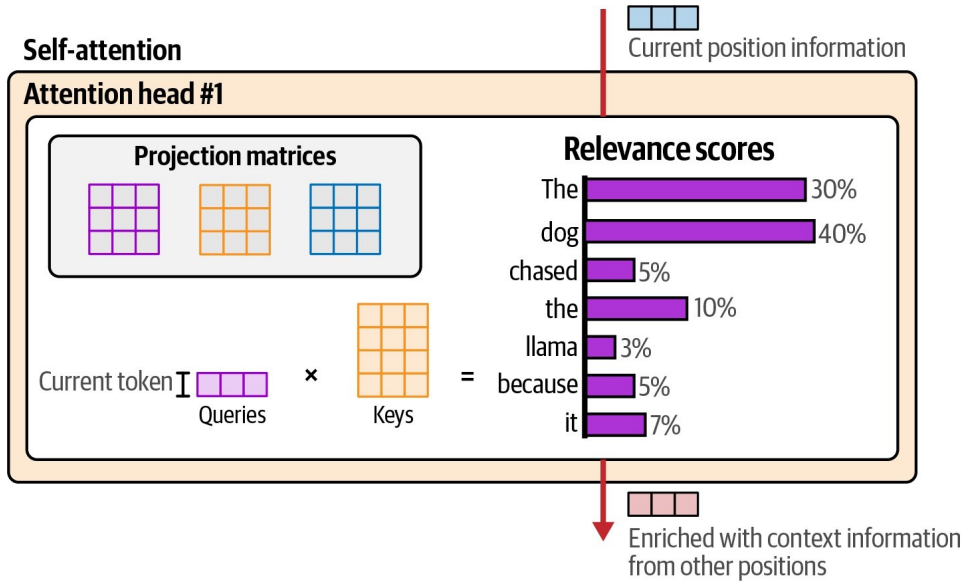
Part 2 begins. How self-attention is actually calculated.

Three projection matrices. Learned in training, they turn the current and previous vectors into queries, keys and values.

Attention head. These matrices, applied together, form one head of the attention layer.

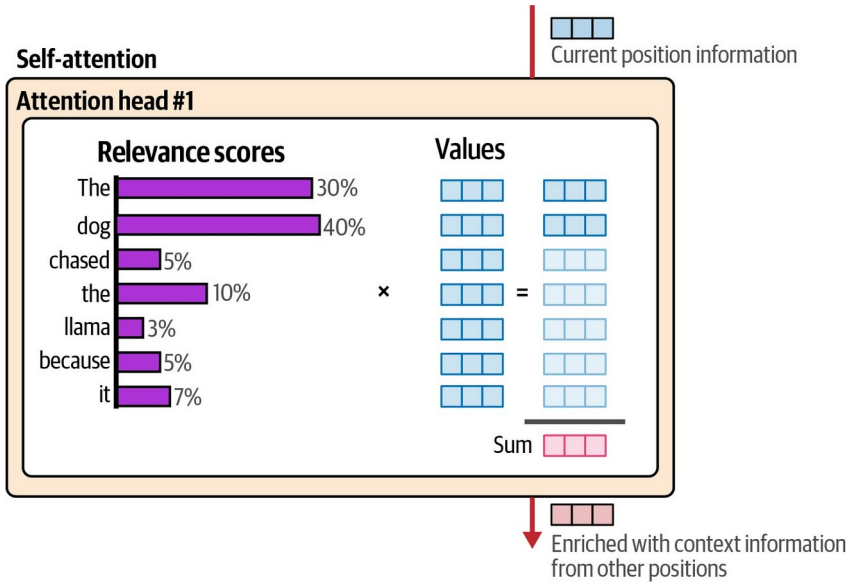
Figure 2-30 · An Illustrated Guide to AI Agents

Scoring relevance, then combining values



Step 1: score

Multiply the current query by every key to get a relevance score per token.

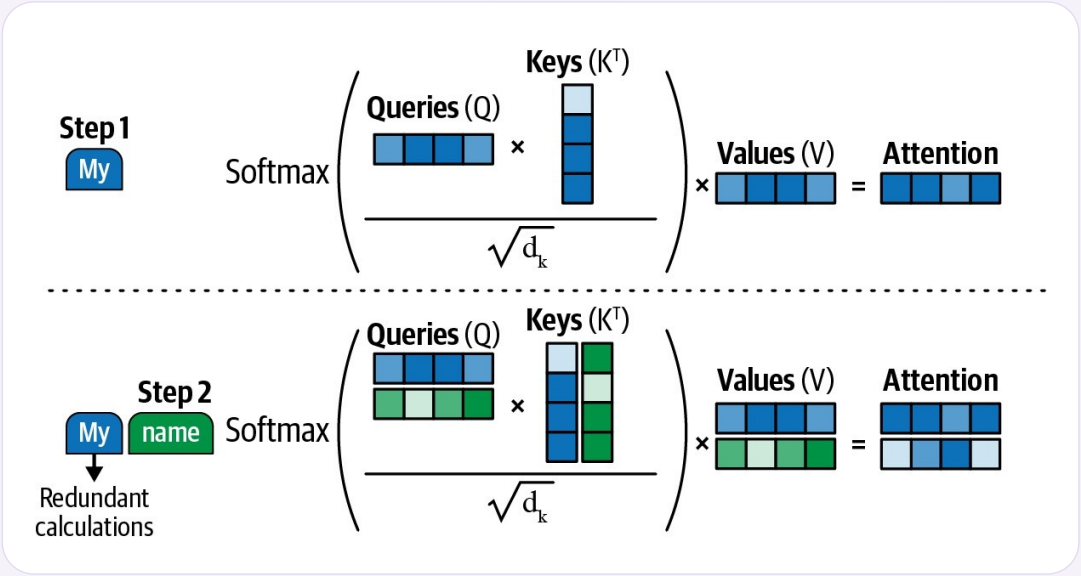


Step 2: combine

Weight each value by its score and sum: the enriched vector for this position.

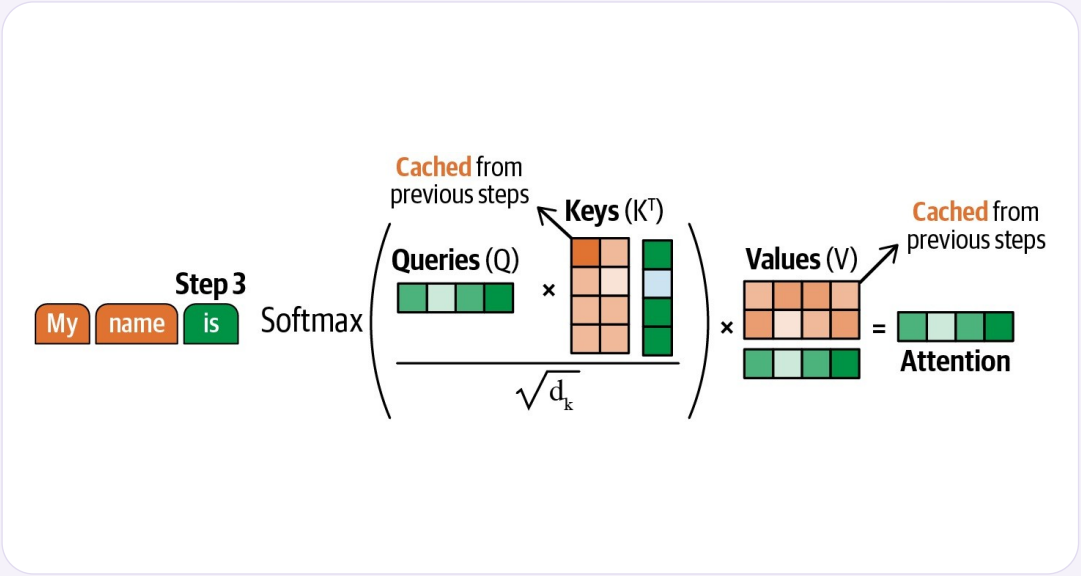
Figures 2-31 and 2-32 · An Illustrated Guide to AI Agents

KV caching revisited



Vanilla attention

Recomputes the keys and values of every previous token at each step: redundant work.

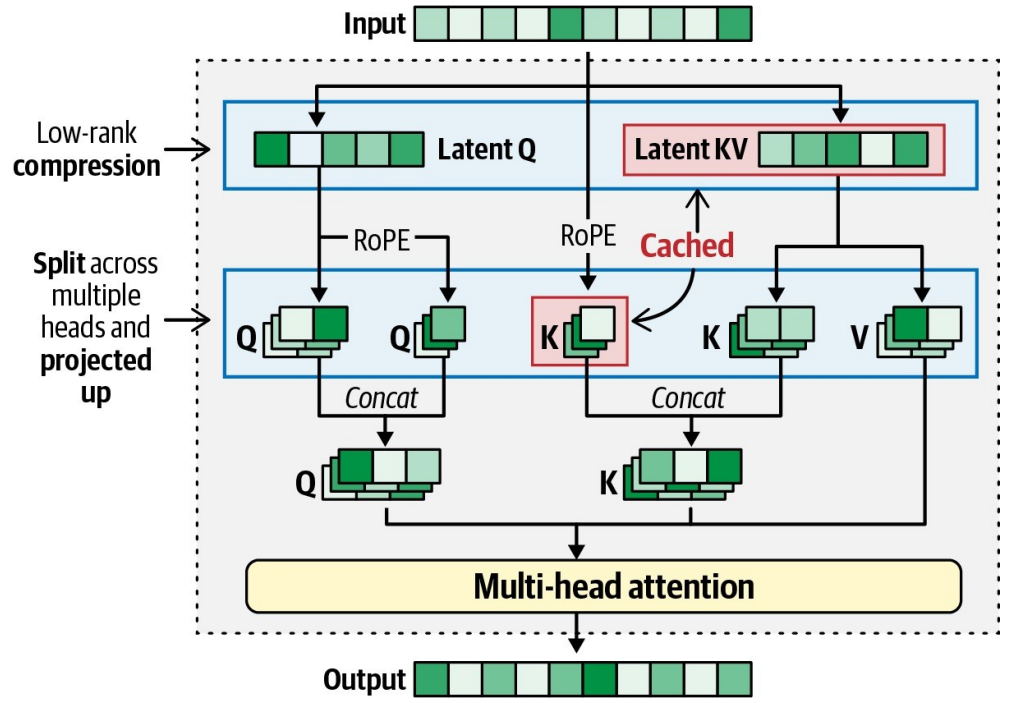


With a KV cache

Reuse stored K and V; only the newest query is needed. Faster, but it costs memory.

Figures 2-34 and 2-35 · An Illustrated Guide to AI Agents

Multi-head Latent Attention (MLA)



The problem. Caching full K and V takes a lot of memory.

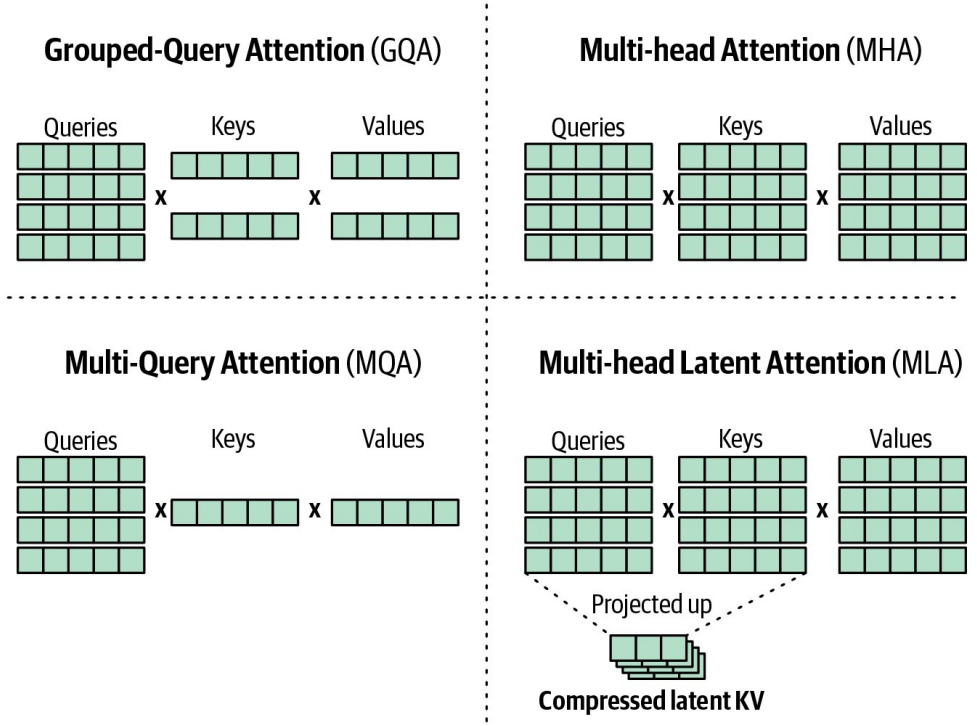
The idea. Compress keys and values into a smaller latent KV, and cache that instead.

The result. A much smaller cache and faster inference, more efficient than grouped-query attention.

First used in DeepSeek-V2, popularized by DeepSeek-R1.

Figure 2-36 · An Illustrated Guide to AI Agents

Four approaches to attention



MHA

Full Q, K and V for every head

GQA and MQA

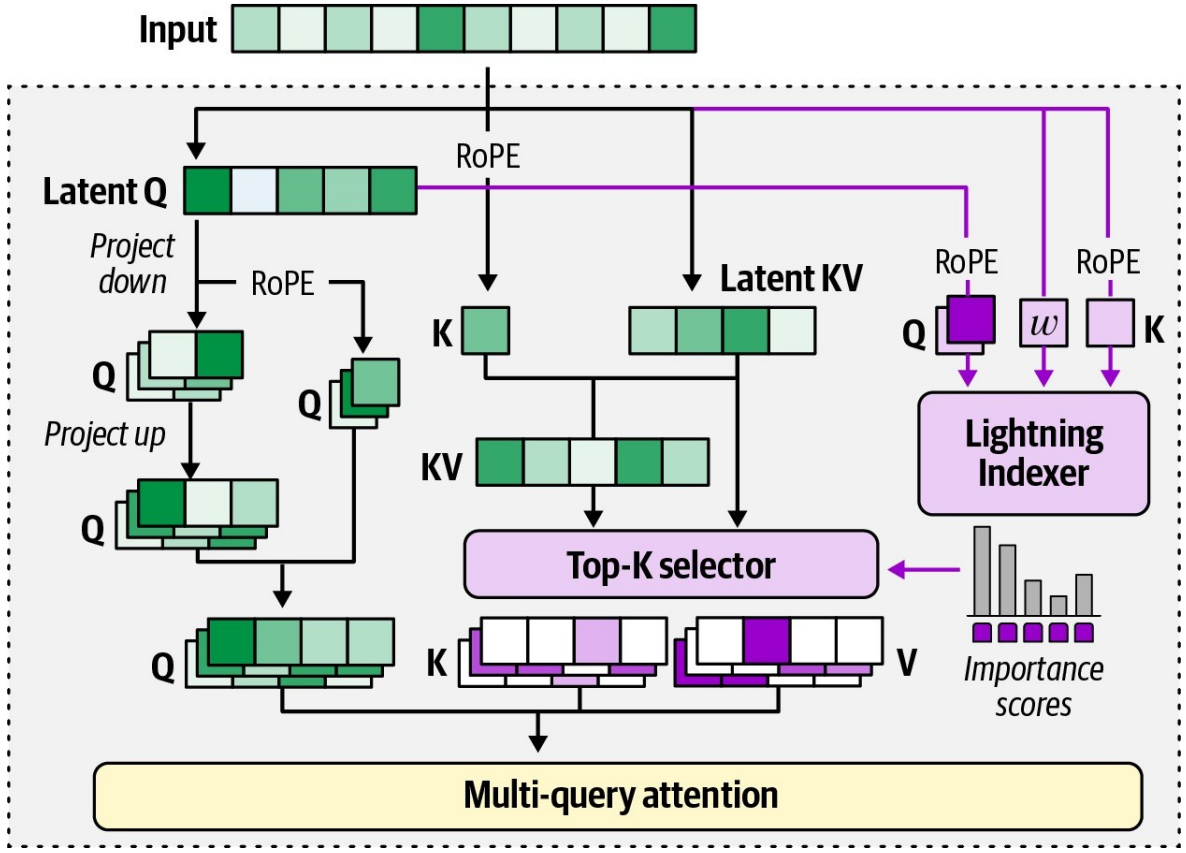
Share K and V across query heads: a smaller cache, but less accurate

MLA

Caches a compressed latent KV

Figure 2-37 · An Illustrated Guide to AI Agents

DeepSeek Sparse Attention (DSA)



Lightning Indexer

Scores how relevant each preceding token is to the current query

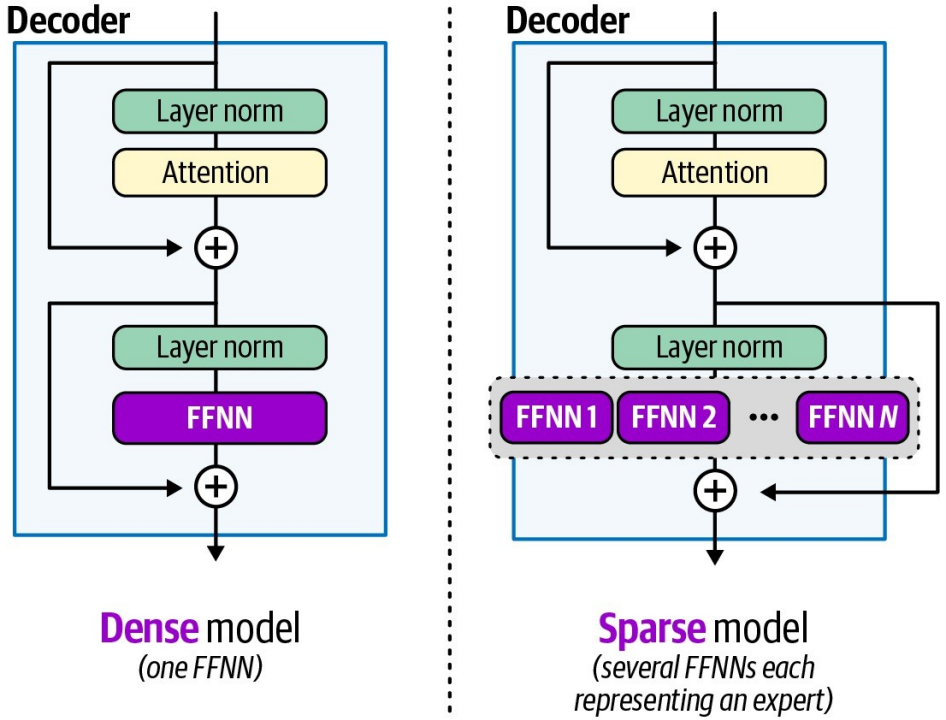
Top-K Selector

Keeps only the highest-scoring KV entries for attention

The paper attends to just 2,048 tokens, drastically cutting computation. Introduced in DeepSeek-V3.2, on top of MLA.

Figure 2-38 · An Illustrated Guide to AI Agents

Mixture of experts: dense versus sparse



Dense. One feed-forward network, active for every token.

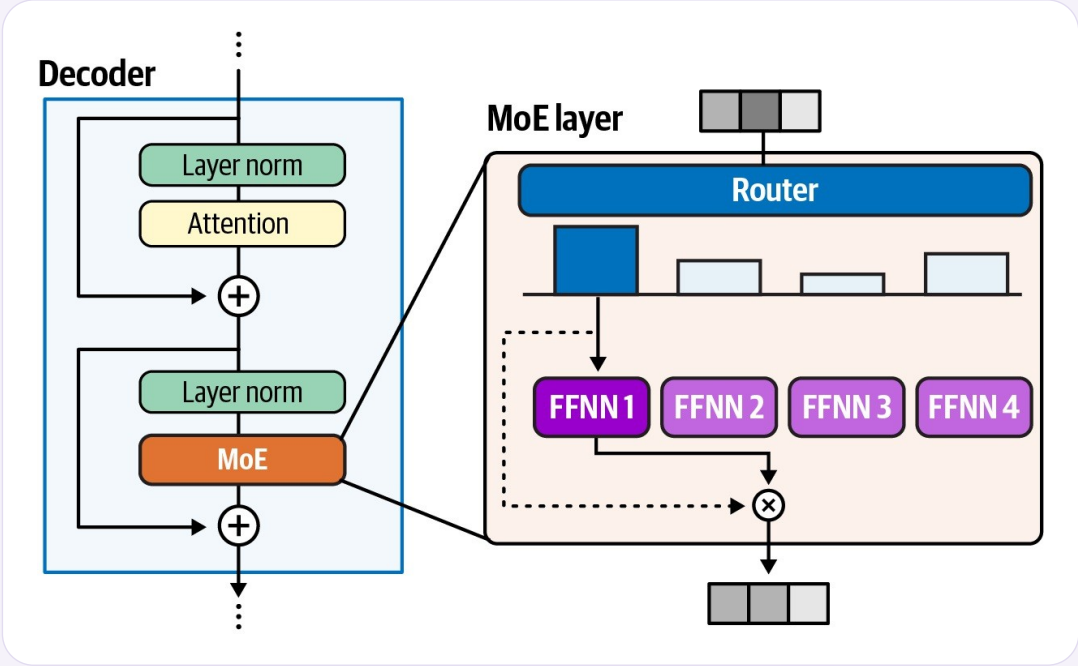
Sparse (MoE). Several smaller feed-forward networks, the “experts”, that together are bigger.

Router. A small network that decides which experts each token goes to.

Experts seem to specialize in fine-grained patterns such as verbs versus numbers.

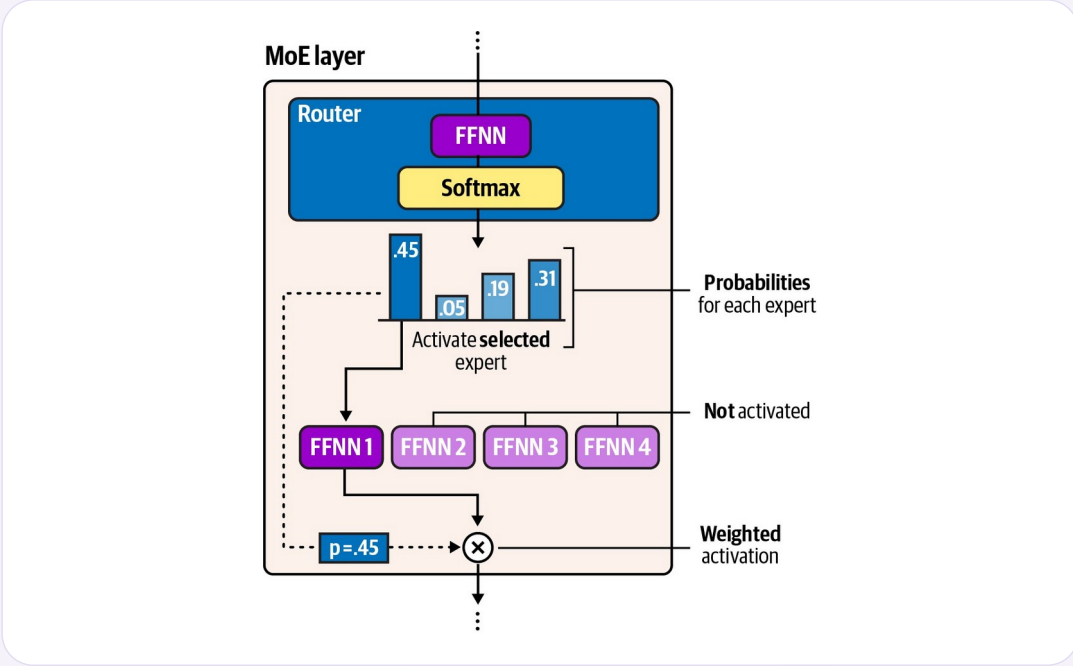
Figure 2-40 · An Illustrated Guide to AI Agents

The router picks the experts



The MoE layer

Router plus experts: each token reaches a small active subset and the rest stay idle.

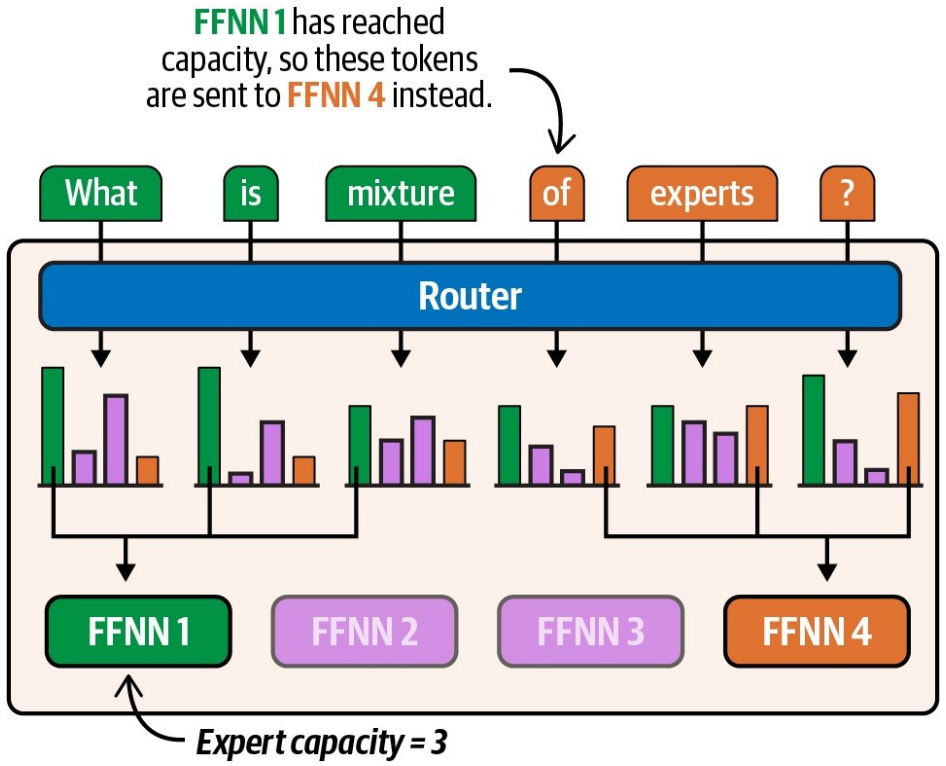


Inside the router

A small network plus softmax gives each expert a probability of being chosen.

Figures 2-41 and 2-42 · An Illustrated Guide to AI Agents

Load balancing across experts



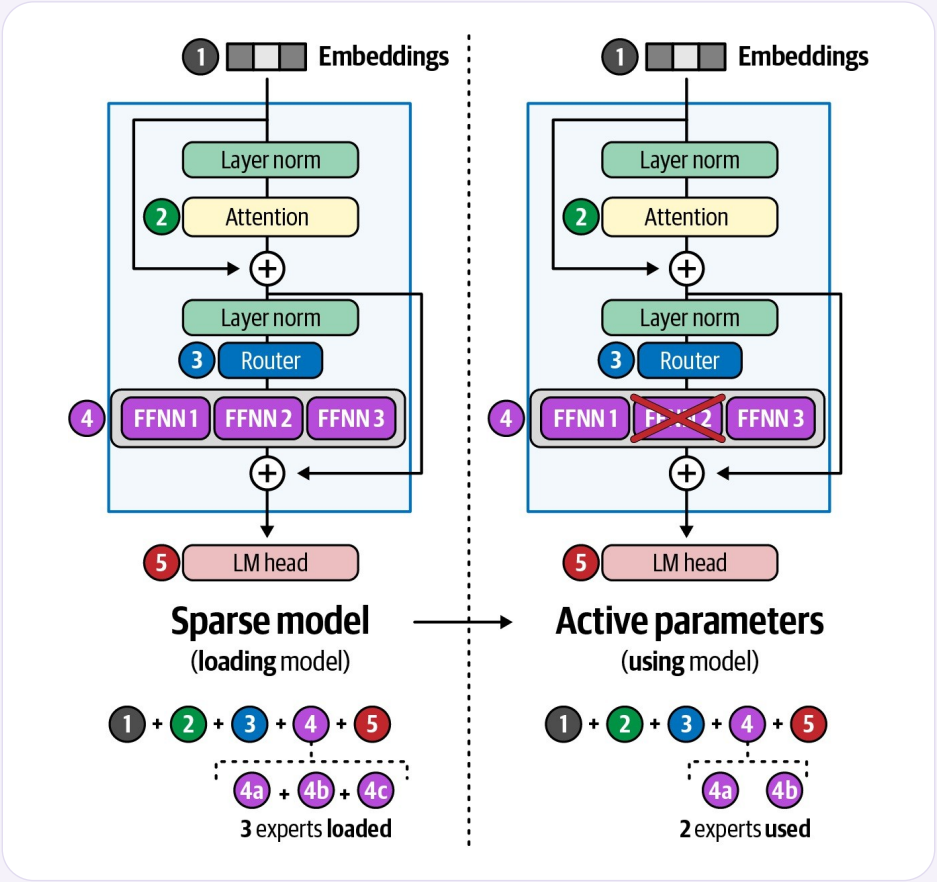
The risk. If the same experts are always chosen, the others are undertrained.

Expert capacity. Cap the tokens each expert takes; overflow goes to the next-best expert.

Router tweaks. Add Gaussian noise before the probabilities, or use an auxiliary loss that rewards even use.

Figure 2-43 · An Illustrated Guide to AI Agents

Sparse versus active parameters



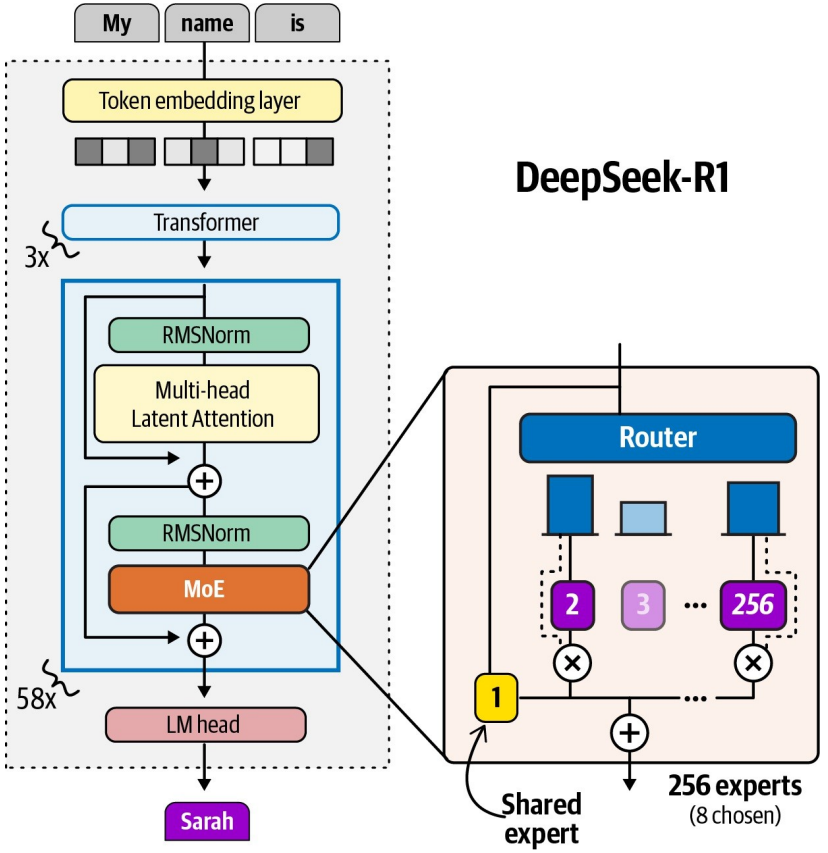
All experts must be loaded into memory, but only a few run per token: not a smaller model, but a much faster one.

Qwen3-30B-A3B: 30 billion parameters loaded, 3 billion active.

Model	Sparse	Active	Experts	Chosen
DeepSeek-R1	671	37	256	8
Qwen3 235B-A22B	235	22	128	8
Kimi-K2	1,000	32	384	8
GPT-OSS 120B	120	5.1	128	4
Llama 4 Maverick	400	17	128	1

Figure 2-44 and Table 2-1 (parameters in billions) · An Illustrated Guide to AI Agents

Example: DeepSeek-R1's MoE layers



256 experts per layer

The router chooses 8 for each token

A shared expert

Bypasses the router and is always active, which tends to hold general knowledge while the others specialize

671B total, 37B active

Memory cost of the whole model, compute cost of a fraction

Figure 2-45 · An Illustrated Guide to AI Agents

Key takeaways

1. LLMs consume and produce tokens; system prompts, multi-turn chats and tool calls let them serve as an agent’s reasoning core.
2. Pre-training builds a base model; SFT and reinforcement learning (RLHF, RLVR with GRPO) shape it to follow instructions and preferences.
3. Tokens flow through blocks of self-attention and feed-forward layers; the LM head scores the next token.
4. For agent developers, context length and the KV cache matter most: they limit input and shape speed and cost.
5. MLA, DSA and mixture of experts cut memory and compute; check total versus active parameters when choosing a model.

Next: Chapter 3, reasoning LLMs